Reviewer Pack — Minimal Order of Hodge Modules and Resolution Proof Width Va…

Entry 0746fcb728e3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 06:30:51 UTC

Minimal Order of Hodge Modules and Resolution Proof Width Variance

Entry ID: `0746fcb728e3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 06:30:51 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Module Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

> The variance in the minimal order of Hodge modules for a set of clauses is lower bounded by Ω(1.5^n log^2 n), where n is the number of variables in the clauses.

Rationale (proposer's reasoning):

> Hodge module theory provides a framework for studying algebraic varieties, and its invariants are expected to reflect deep structural properties. Since resolution proof width is a measure of the difficulty of a SAT instance, a correlation between these two areas could reveal new insights into both complexity theory and algebraic geometry.

Taxonomy category: `algebraic_geometry` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `bd5eec11632929d5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The variance in minimal order of Hodge modules for clauses with n variables is considered supported if it falls within Ω(1.5^n log^2 n) and falsified if any seed produces a variance below this threshold.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge module theory" AND "resolution proof complexity"` - `"minimal order Hodge modules" AND "resolution proof width variance"` - `"Algebraic Geometry" AND "resolution proof complexity" WITHIN 1000 miles OF "Hodge Module Theory"`

Top relevant hits considered: - [http://arxiv.org/abs/math/9904032v1] Recent Trends in Algebraic Geometry – EuroConference on Algebraic Geometry (Warwick, July 1996) - [http://arxiv.org/abs/math/9806006v1] Special Langrangian geometry and slightly deformed algebraic geometry (spLag and sdAG) - [http://arxiv.org/abs/2112.12010v1] Algebraic geometry in mixed characteristic

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_clauses(n, num_clauses):
        clauses = []
        for _ in range(num_clauses):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for i in range(n) for j in range(i+1, n)):
                clauses.append(clause)
        return clauses

    def hodge_module_order(clause):
        # Simplified model of Hodge module order
        return abs(sum(clause)) + len(clause)

    def resolution_width(clause):
        stack = []
        for literal in clause:
            if literal == 1:
                stack.append(literal)
            elif literal == -1:
                if stack and stack[-1] == -literal:
                    stack.pop()
                else:
                    return float('inf')
        return len(stack)
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    variances = []

    for n in n_values:
        clauses = generate_clauses(n, 30)
        hodge_orders = [hodge_module_order(clause) for clause in clauses]
        widths = [resolution_width(clause) for clause in clauses]

        if not hodge_orders or not widths:
            return {
                "metric_name": "variance",
                "metric_value": None,
                "instances_tested": 0,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }

        variance = sum((x - (sum(hodge_orders) / len(hodge_orders))) ** 2 for x in hodge_orders) /
        variances.append(variance)

    mean_variance = sum(variances) / len(variances)
    conjecture_holds = all(v >= 1.5**n * math.log(n)**2 for n, v in zip(n_values, variances))
    counterexample = "" if conjecture_holds else "variance_below_threshold"

    return {
        "metric_name": "variance",
        "metric_value": mean_variance,
        "instances_tested": 30 * len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(rang
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_variance = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_variance} std=0.0 support_fraction={support_fraction}
    elif any(not r["conjecture_holds"] and r["counterexample"] == "variance_below_threshold" for r
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"variance_below_threshold\" first_failing_seed={
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
L: {'metric_name': 'variance', 'metric_value': 2079.931111111111, 'instances_tested': 180, 'n_max': 
TRIAL: {'metric_name': 'variance', 'metric_value': 2604.7599999999998, 'instances_tested': 180, 'n_m
TRIAL: {'metric_name': 'variance', 'metric_value': 1825.3096296296299, 'instances_tested': 180, 'n_m
TRIAL: {'metric_name': 'variance', 'metric_value': 2901.2792592592596, 'instances_tested': 180, 'n_m
TRIAL: {'metric_name': 'variance', 'metric_value': 2330.3762962962965, 'instances_tested': 180, 'n_m
TRIAL: {'metric_name': 'variance', 'metric_value': 2280.3088888888888, 'instances_tested': 180, 'n_m
TRIAL: {'metric_name': 'variance', 'metric_value': 2385.3044444444445, 'instances_tested': 180, 'n_m
TRIAL: {'metric_name': 'variance', 'metric_value': 2551.571851851852, 'instances_tested': 180, 'n_ma

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81c0d791.py", line 94, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81c0d791.py", line 94, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError:
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper assessment of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14308
2	preregistration	ollama_remote	glm4:latest	0	0	8941
3	novelty	ollama_remote	glm4:latest	0	0	8118
4	novelty	ollama_remote	glm4:latest	0	0	9486
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20155
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9420
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10699
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9963
9	judge	ollama_remote	glm4:latest	0	0	12421

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 103511 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0746fcb728e3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0746fcb728e3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0746fcb728e3.tar.gz` (if generated)